Puente: donde estamos

Los problemas de la linealidad

Regresion polinomial

Polinomios truncados y base de funciones

Splines de regresion

Suavizamiento spline

Modelos Aditivos Generalizados (GAMs)

Sintesis y cierre

Referencias

Análisis Avanzado de Datos

Sesión 3: Más allá de la linealidad

Nicolás López

Primer semestre de 2026

1. Puente: donde estamos
2. Los problemas de la linealidad
3. Regresion polinomial
4. Polinomios truncados y base de funciones
5. Splines de regresion
6. Suavizamiento spline
7. Modelos Aditivos Generalizados (GAMs)
8. Sintesis y cierre
9. Referencias

Puente: donde estamos

Los problemas de la linealidad

Regresion polinomial

Polinomios truncados y base de funciones

Splines de regresion

Suavizamiento spline

Modelos Aditivos Generalizados (GAMs)

Sintesis y cierre

Referencias

Puente: donde estamos

Lo que hemos construido (Sesiones 1 y 2)

Sesion 1 – Regularizacion:

- Regresion lineal $Y = X\beta + \epsilon$ estimada por MCO.
- Con $p \gg n$: $X^T X$ es singular (multicolinealidad).
- Solucion: penalizacion Ridge (L_2) y Lasso (L_1).
- Trade-off sesgo–varianza: introducir sesgo para reducir varianza.

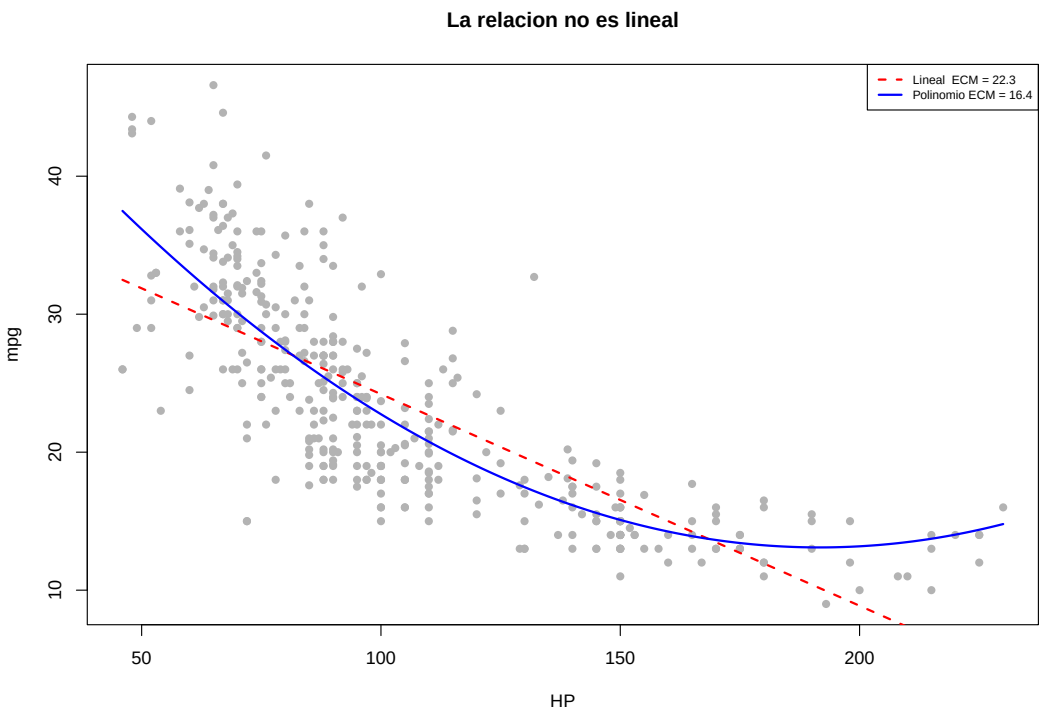
Sesion 2 – Validacion:

- ECM_{Train} es optimista; ECM_{Test} mide generalizacion.
- Metodos: validacion simple, LOOCV, k-fold, Bootstrap.
- El hiperparametro λ se selecciona minimizando ECM_{Test}.

Limitacion central de ambas sesiones

Ridge y Lasso controlan la **magnitud** de β, pero siguen asumiendo $E(Y \mid X) = X\beta$: el efecto de cada variable sobre Y es una **linea recta**.

El problema que persiste



El ECM cae de 22.3 a 16.4 con el termino cuadratico. La curvatura en los datos es evidente.

Conexion con el Taller 1

Los estudiantes trabajan con 5000 genes y 1200 lineas celulares para predecir la efectividad de un tratamiento anticancer.

- **Preguntas 1–5**: multicolinealidad y seleccion de λ (S1 y S2).
- **Preguntas 6–8**: modelo final. Los coeficientes Lasso asumen que el efecto de cada gen sobre la respuesta es **lineal**.

Reflexion: Lasso puede seleccionar un gen con coeficiente no nulo. Ese coeficiente es la mejor aproximacion lineal al efecto real del gen. Si el efecto verdadero es curvo, el modelo lineal lo representa mal.

Esta sesion muestra como relajar ese supuesto. Mas adelante se discutira como combinarlo con alta dimension.

Puente: donde estamos

Los problemas de la linealidad

Regresion polinomial

Polinomios truncados y base de funciones

Splines de regresion

Suavizamiento spline

Modelos Aditivos Generalizados (GAMs)

Sintesis y cierre

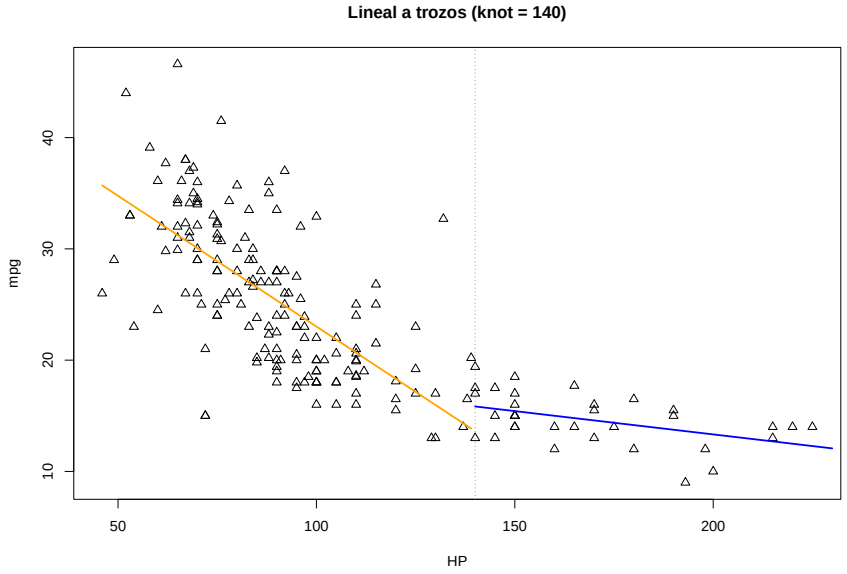
Referencias

Los problemas de la linealidad

Regresion lineal a trozos

Una primera idea: ajustar rectas distintas en cada region de x.

```
mod11 <- lm(mpg ~ horsepower,
  data = Auto[train,] %>% filter(horsepower < 140))
mod12 <- lm(mpg ~ horsepower,
  data = Auto[train,] %>% filter(horsepower >= 140))
plot(Auto$horsepower[train], Auto$mpg[train],
     pch = 2, xlab = "HP", ylab = "mpg",
     main = "Lineal a trozos (knot = 140)")
lines(xvals$horsepower[xvals$horsepower <  140],
  predict(mod11, xvals %>% filter(horsepower <  140)),
  lwd = 2, col = "orange")
lines(xvals$horsepower[xvals$horsepower >= 140],
  predict(mod12, xvals %>% filter(horsepower >= 140)),
  lwd = 2, col = "blue")
abline(v = 140, lty = 3, col = "grey50")
```



Problemas fundamentales

ECM prueba (lineal a trozos): 17.26. No mejora respecto al modelo lineal global (22.28).

Problema 1 – Arbitrariedad del knot: El valor $k = 140$ fue elegido sin criterio formal. Otro analista elegiria $k = 100$ con resultados distintos.

Problema 2 – Discontinuidad: La funcion tiene un salto en $k = 140$, biologicamente poco plausible: el efecto de un gen raramente cambia de forma abrupta al cruzar un umbral exacto.

> *Ridge y Lasso controlan la magnitud de β, pero no resuelven la no-linealidad ni la discontinuidad subyacente.*

Puente: donde estamos

Los problemas de la linealidad

Regresion polinomial

Polinomios truncados y base de funciones

Splines de regresion

Suavizamiento spline

Modelos Aditivos Generalizados (GAMs)

Sintesis y cierre

Referencias

Regresion polinomial

El modelo polinomial global

La generalizacion natural del modelo lineal es el polinomio global:

$$y_i = \beta_0 + \beta_1 x_i + \cdots + \beta_d x_i^d + \epsilon_i$$

Puede escribirse como RLM con covariables $\phi_j(x) = x^j$; se estima directamente por MCO. **No requiere ninguna tecnica nueva.**

poly() ortogonal vs. raw

En R, `poly(x, d)` genera polinomios **ortogonales**, no los coeficientes de $x, x^2, \ldots, x^d$ directamente.

Problema que resuelve: cuando x es, por ejemplo, HP (valores entre 46 y 230), las columnas de la matriz de diseno son $x, x^2, x^3, \ldots$, con valores que van de 46 a 12 167 000. Estas columnas son casi colineales entre si, lo que hace que $X^T X$ este mal condicionada y los estimadores sean inestables.

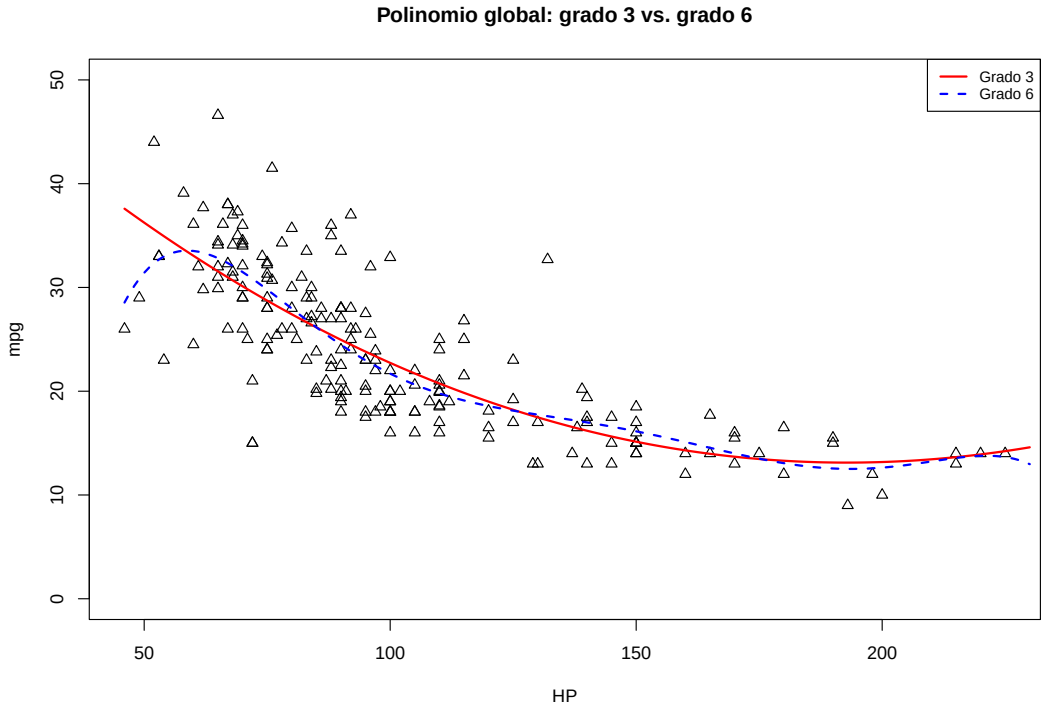
Solucion – polinomios ortogonales: `poly(x, d)` construye d funciones que son *perpendiculares* entre si ($\langle \phi_j, \phi_k \rangle = 0$ para $j \neq k$). La matriz de diseno resultante esta mucho mejor condicionada.

Cuando usar `poly(x, d, raw=TRUE)`: si necesita interpretar directamente los coeficientes como los de la ecuacion $\beta_0 + \beta_1 x + \beta_2 x^2 + \ldots$. En la practica se usa para presentacion de resultados, no para ajuste.

```
mod_lm2 <- lm(mpg ~ poly(horsepower,2), data=Auto[train,])
mod_lm3 <- lm(mpg ~ poly(horsepower,3), data=Auto[train,])
mod_lm6 <- lm(mpg ~ poly(horsepower,6), data=Auto[train,])
mse_fn <- function(m)
  mean((Auto$mpg[test]-predict(m,Auto)[test])^2)
round(c(d2=mse_fn(mod_lm2),
        d3=mse_fn(mod_lm3),
        d6=mse_fn(mod_lm6)),2)
```

```
##    d2    d3    d6
## 16.38 16.36 17.19
```

Fenomeno de Runge: sobreajuste en los extremos



El polinomio de grado 6 oscila erráticamente en los extremos (**fenomeno de Runge**): los polinomios globales de alto grado no son localmente adaptativos. Se recomienda $d \leq 4$.

Polinomio local de grado 2

Ajustar polinomios de **bajo grado localmente** es mas estable:

```
mod11_2 <- lm(mpg ~ poly(horsepower,2),
  data=Auto[train,] %>% filter(horsepower < 140))
mod12_2 <- lm(mpg ~ poly(horsepower,2),
  data=Auto[train,] %>% filter(horsepower >= 140))
mse_local2 <- mean(c(
  ((Auto[test,] %>% filter(horsepower <  140))$mpg -
     predict(mod11_2, Auto[test,] %>%
               filter(horsepower <  140)))^2,
  ((Auto[test,] %>% filter(horsepower >= 140))$mpg -
     predict(mod12_2, Auto[test,] %>%
               filter(horsepower >= 140)))^2))
round(c(global_d2=mse_fn(mod_lm2), local_d2=mse_local2), 2)
```

```
## global_d2  local_d2
##     16.38     16.35
```

Menor ECM que el global, pero la **discontinuidad en** $k = 140$ **persiste**. Necesitamos garantizar continuidad en los knots.

Puente: donde estamos

Los problemas de la linealidad

Regresion polinomial

Polinomios truncados y base de funciones

Splines de regresion

Suavizamiento spline

Modelos Aditivos Generalizados (GAMs)

Sintesis y cierre

Referencias

Polinomios truncados y base de funciones

La solucion a la discontinuidad

Para un knot k, el **polinomio truncado de grado** h es:

$$g^h(x,k) = (x-k)_+^h = \begin{cases} (x-k)^h & x \geq k \\ 0 & x < k \end{cases}$$

Agregar $g^h(x,k)$ al polinomio base garantiza que la funcion y sus primeras $h-1$ derivadas coincidan a ambos lados de k.

Grado h y suavidad en k

- $h = 1$: f continua en k (sin salto).
- $h = 2$: f y f' continuas (sin quiebre de pendiente).
- $h = 3$: f, f' y f'' continuas – **spline cubico**.

Verificacion algebraica: spline lineal ($h = 1$)

Con un knot k, el modelo lineal continuo es:

$$y_i = \beta_0 + \beta_1 x_i + \beta_2 (x_i - k)_+ + \epsilon_i$$

Expandiendo por casos y evaluando en $x = k$:

$$\text{Izq: } \beta_0 + \beta_1 k \qquad \text{Der: } (\beta_0 - \beta_2 k) + (\beta_1 + \beta_2)k = \beta_0 + \beta_1 k$$

Los dos valores coinciden. **Continuidad garantizada con 3 parametros.** El modelo a trozos necesita 4 parametros y no garantiza continuidad.

Spline cubico con polinomios truncados

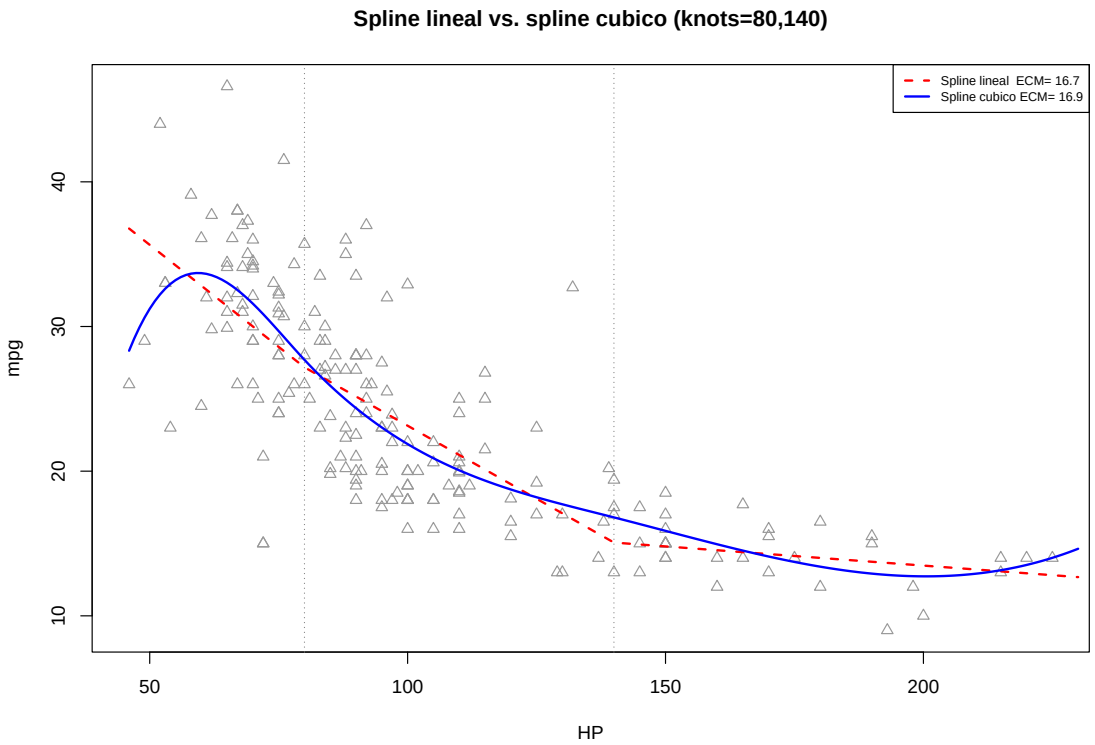
$$y_i = \underbrace{\beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3}_{\text{cubico global}} + \underbrace{\beta_4 (x - k_1)_+^3 + \beta_5 (x - k_2)_+^3}_{\text{ajuste local}} + \epsilon_i$$

Seis parametros totales. La funcion es continua y suave en k_1 y k_2: las tres primeras derivadas coinciden a ambos lados de cada knot.

```
mod1_knot <- lm(
  mpg ~ horsepower +
    I((horsepower-80)*(horsepower>=80)) +
    I((horsepower-140)*(horsepower>=140)),
  data=Auto[train,])
mod2_knot <- lm(
  mpg ~ horsepower+I(horsepower^2)+I(horsepower^3)+
    I(((horsepower-80)*(horsepower>=80))^3)+
    I(((horsepower-140)*(horsepower>=140))^3),
  data=Auto[train,])
mse1_knot <- mean((Auto$mpg[test]-predict(mod1_knot,Auto)[test])^2)
mse2_knot <- mean((Auto$mpg[test]-predict(mod2_knot,Auto)[test])^2)
round(c(lineal=mse1_knot, cubico=mse2_knot),2)
```

```
## lineal cubico
##  16.69  16.93
```

El spline cubico en accion



La suavidad del spline cubico proviene de la continuidad de f'' (curvatura), que elimina los quiebres angulares del spline lineal.

La perspectiva unificadora: base de funciones

Todos los modelos anteriores son casos de un **modelo lineal en una base de funciones**:

$$y_i = \sum_{j=1}^{p} \beta_j \phi_j(x_i) + \epsilon_i$$

Como ϕ_j es conocida, esto es RLM estimado por MCO. La eleccion de $\{\phi_j\}$ determina las propiedades del ajuste.

Base	Funciones ϕ_j
Polinomial	$1,\ x,\ x^2,\ x^3$
Truncada (cub.)	$1, x, x^2, x^3, (x-k_1)_+^3, (x-k_2)_+^3$
B-spline	Polinomios locales, valores entre 0 y 1
Fourier	$1, \sin(x), \cos(x), \sin(2x), \cos(2x), \ldots$

Pregunta clave: *la base truncada y la B-spline representan* ***exactamente las mismas funciones****. ¿Por que entonces usar B-splines? La respuesta es estabilidad numerica.*

Puente: donde estamos

Los problemas de la linealidad

Regresion polinomial

Polinomios truncados y base de funciones

Splines de regresion

Suavizamiento spline

Modelos Aditivos Generalizados (GAMs)

Sintesis y cierre

Referencias

Splines de regresion

Definicion formal del spline de regresion

Definicion

Un **spline de regresion de grado** d **con** K **knots** $k_1 < k_2 < \cdots < k_K$ es una funcion $f(x)$ tal que:

1. En cada region entre knots consecutivos, f es un **polinomio de grado** d.
2. En cada knot k_j, las primeras $d-1$ derivadas de f son **continuas**.

El numero de parametros (grados de libertad) es $df = d + 1 + K$.

Para un spline cubico ($d = 3$) con K knots: $df = 4 + K$.

La estimacion se reduce a MCO eligiendo cualquier base $\{\phi_j\}$ que genere este espacio de funciones.

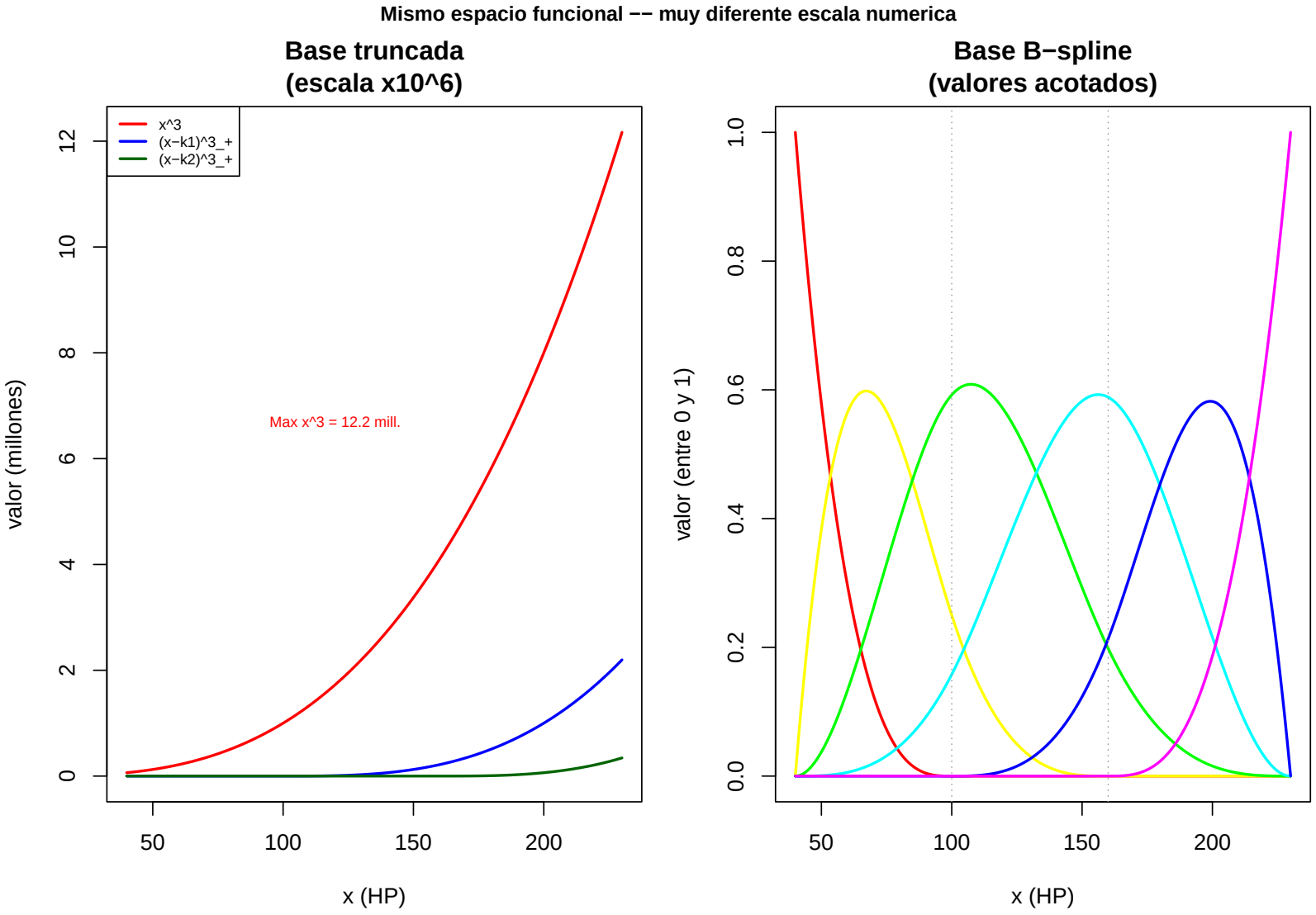
Mismo espacio funcional: la analogia del Lego

Mismo espacio funcional significa: cualquier curva que puedas construir con la base truncada, puedes construirla exactamente igual con la base B-spline. Solo cambian los coeficientes β.

Piensalo como **dos juegos de Lego distintos** para construir la misma figura:

- **Lego truncado:** el juego tiene piezas enormes (x^3 con $x = 200$ da $8\,000\,000$). Puedes construir la figura, pero las piezas son tan grandes y mal balanceadas que se tambalean y el ensamblaje es impreciso.
- **Lego B-spline:** el juego tiene piezas estandarizadas de tamano entre 0 y 1. La figura final es **identica**, pero encajan perfectamente y el ensamblaje es robusto.
  *El modelo ajustado es el mismo. La diferencia es la **estabilidad numerica** del proceso de estimacion.*

Visualizacion: la escala importa



El numero de condicion: que mide exactamente

El **numero de condicion** de una matriz A es:

$$\kappa(A) = \frac{\sigma_{\max}(A)}{\sigma_{\min}(A)}$$

donde $\sigma_{\max}$ y $\sigma_{\min}$ son el mayor y menor valor singular de A.

Que mide en la practica: cuanto se amplifica el error al resolver el sistema $A\hat{\beta} = b$. Si cometo un error relativo de ε al medir b (por redondeo del computador, que tiene precision $\approx 10^{-16}$), el error relativo en $\hat{\beta}$ puede ser hasta $\kappa(A) \cdot \varepsilon$.

Ejemplo concreto: - $\kappa = 10^3$: un error de redondeo 10^{-16} produce error en $\hat{\beta}$ de $\approx 10^{-13}$. Inofensivo. - $\kappa = 10^{15}$: el mismo error de redondeo produce error de $\approx 10^{-1} \approx 10\%$ en $\hat{\beta}$. Los coeficientes estimados son **completamente sin sentido**.

Numero de condicion: base truncada vs. B-spline

```
x_hp  <- sort(Auto$horsepower[train])
k1c <- 80; k2c <- 140

# Matriz de diseno: base truncada
X_t <- cbind(1, x_hp, x_hp^2, x_hp^3,
  ((x_hp-k1c)*(x_hp>=k1c))^3,
  ((x_hp-k2c)*(x_hp>=k2c))^3)

# Matriz de diseno: B-spline
X_b <- cbind(1, bs(x_hp, knots = c(k1c,k2c)))

cat("Cond. base truncada:",
    formatC(kappa(X_t), format="e", digits=2), "\n")
```

```
## Cond. base truncada: 4.95e+08
```

```
cat("Cond. B-spline:     ",
    formatC(kappa(X_b), format="e", digits=2), "\n")
```

```
## Cond. B-spline:      3.03e+01
```

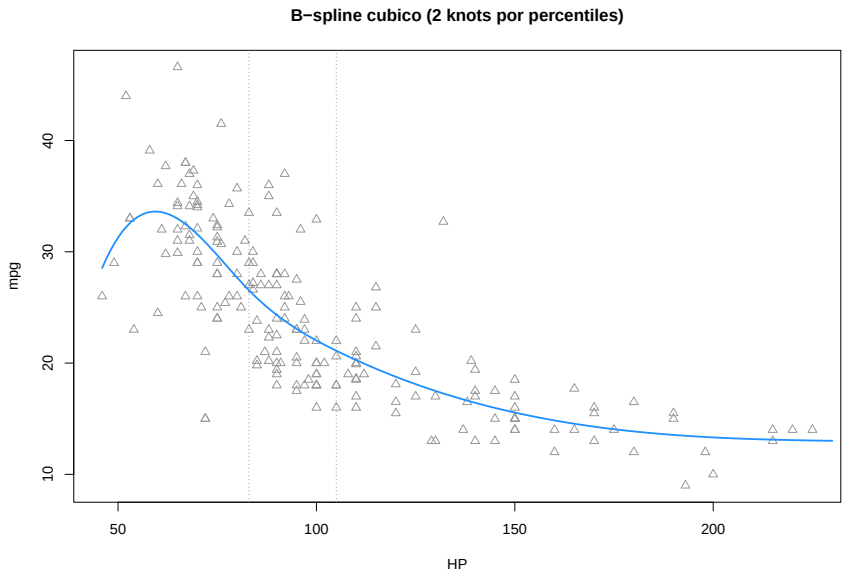
La base B-spline es ordenes de magnitud mas estable. **El ajuste final es identico; la aritmetica es distinta.**

Uso de bs() en R

```
library(splines)
knots_q <- quantile(Auto$horsepower[train],
                    probs = c(1/3, 2/3))
mod_bs <- lm(mpg ~ bs(horsepower, knots=knots_q),
             data=Auto[train,])
mse_bs <- mean(
  (Auto$mpg[test]-predict(mod_bs,Auto)[test])^2)
cat("ECM:", round(mse_bs,2),
    " | df:", length(coef(mod_bs))-1, "\n")
```

```
## ECM: 16.82  | df: 5
```

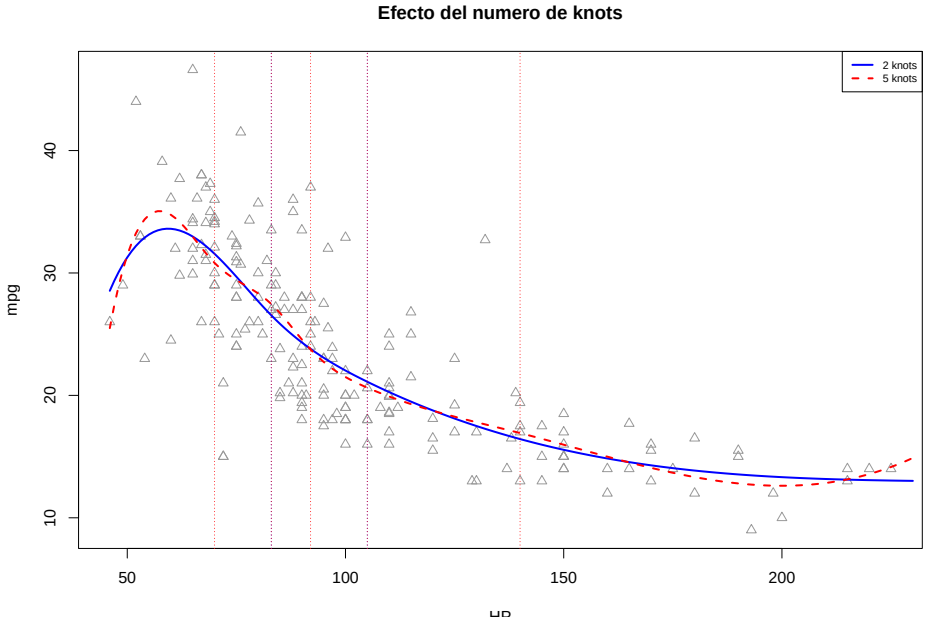
```
plot(Auto$horsepower[train], Auto$mpg[train],
     pch=2, col="grey60", xlab="HP", ylab="mpg",
     main="B-spline cubico (2 knots por percentiles)")
lines(xvals$horsepower, predict(mod_bs,xvals),
      lwd=2, col="dodgerblue")
abline(v=knots_q, lty=3, col="grey50")
```



Efecto del numero de knots

A mayor numero de knots, mayor flexibilidad, pero mayor riesgo de sobreajuste.

```
knots2 <- quantile(Auto$horsepower[train], c(1/3,2/3))
knots5 <- quantile(Auto$horsepower[train],
                   seq(1/6,5/6,by=1/6))
mod_bs2 <- lm(mpg~bs(horsepower,knots=knots2),
              data=Auto[train,])
mod_bs5 <- lm(mpg~bs(horsepower,knots=knots5),
              data=Auto[train,])
plot(Auto$horsepower[train], Auto$mpg[train],
     pch=2, col="grey60", xlab="HP", ylab="mpg",
     main="Efecto del numero de knots")
lines(xvals$horsepower, predict(mod_bs2,xvals),
      lwd=2, col="blue")
lines(xvals$horsepower, predict(mod_bs5,xvals),
      lwd=2, col="red", lty=2)
abline(v=knots2, lty=3, col="blue", lwd=0.8)
abline(v=knots5, lty=3, col="red",  lwd=0.8)
legend("topright", legend=c("2 knots","5 knots"),
       col=c("blue","red"), lty=c(1,2), lwd=2, cex=0.7)
```



Splines naturales: definicion y motivacion

Un **spline natural** (NS) es un spline cubico que impone una restriccion adicional: la funcion debe ser **lineal mas alla de los knots extremos de los datos**.

Por que esto importa: en los extremos del dominio hay pocos datos. Un spline cubico libre puede oscilar erráticamente en esas regiones, produciendo predicciones extrapoladas absurdas. El spline natural fuerza linealidad (sin curvatura, $f'' = 0$) en esas regiones: mas conservador y realista para extrapolacion.

Consecuencia en df

La condicion de linealidad fuera de los extremos impone **2 restricciones** sobre el espacio de funciones: una restriccion por cada extremo del dominio de datos. Por lo tanto $df_{\text{Natural}} = df_{\text{B-spline}} - 2$.

Por que se pierden exactamente 2 df y no 4

Intuicion: un polinomio cubico $a + bx + cx^2 + dx^3$ es lineal cuando $c = 0$ y $d = 0$: dos condiciones. Con dos extremos, ¿no deberiamos perder $2 \times 2 = 4$ df?

Por que no: en un spline, la continuidad de f'' en los knots ya vincula los coeficientes entre piezas adyacentes. El trozo extremo izquierdo esta casi completamente determinado por el trozo interior (debe coincidir en f, f', f'' en el knot de frontera). Le queda **un solo grado de libertad**: su coeficiente cubico. Imponer $f''(\text{frontera}) = 0$ elimina ese 1 parametro libre.

Lo mismo ocurre en el extremo derecho: 1 parametro libre, 1 restriccion.

Total: $1 + 1 = 2$ **restricciones** $\Rightarrow$ **2 df menos.**

Los 2 df perdidos: conteo explicito en R

Con $K = 2$ knots interiores:

```
k_int <- c(80, 140)

# B-spline cubico: K + 3 parametros
ncol_bs <- ncol(bs(Auto$horsepower,  knots=k_int))

# Spline natural: K + 1 parametros = (K+3) - 2
ncol_ns <- ncol(ns(Auto$horsepower, knots=k_int))

cat("Columnas B-spline (K+3):", ncol_bs, "\n")
```

```
## Columnas B-spline (K+3): 5
```

```
cat("Columnas Natural  (K+1):", ncol_ns, "\n")
```

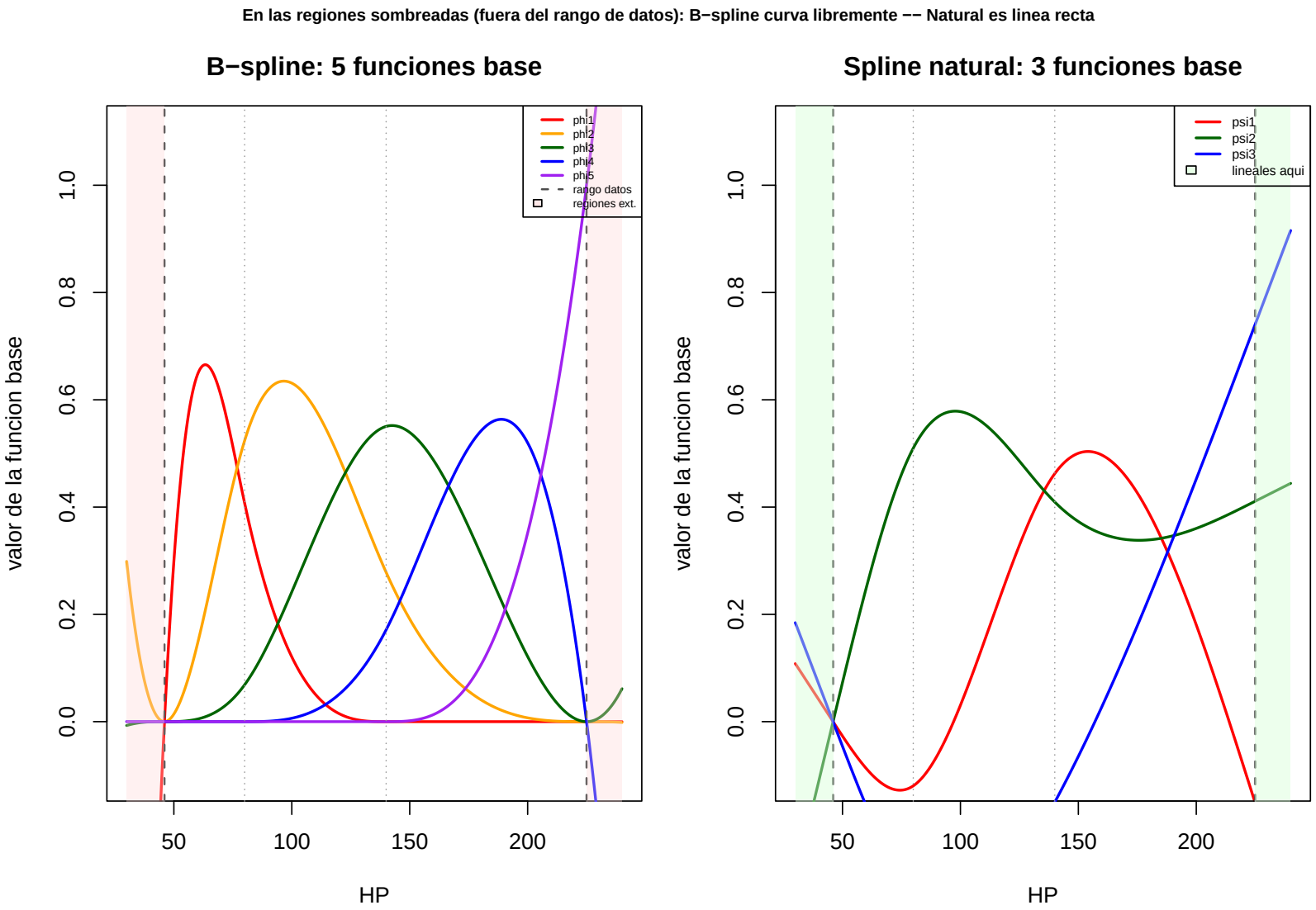
```
## Columnas Natural  (K+1): 3
```

```
cat("Diferencia:             ", ncol_bs - ncol_ns,
    "parametros\n")
```

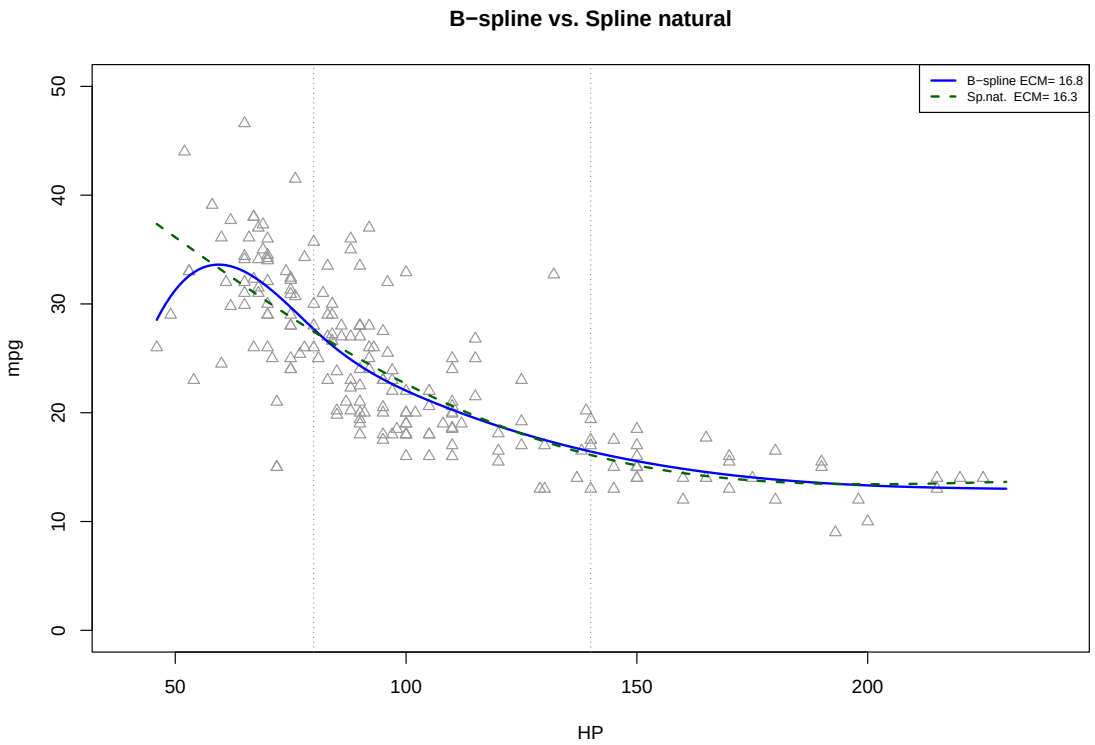
```
## Diferencia:              2 parametros
```

Las 2 columnas que desaparecen son las responsables de la curvatura en las regiones extremas del dominio de x. Con el spline natural, esas regiones son forzadas a ser lineas rectas.

Visualizacion: 5 funciones bs vs. 3 funciones ns



B-spline vs. spline natural: ajuste comparado



En los extremos el spline natural es mas conservador. Preferible para prediccion fuera del rango de datos observados.

Que significan los grados de libertad (df)

Los **grados de libertad** = numero de parametros que el modelo puede ajustar libremente para acercarse a los datos.

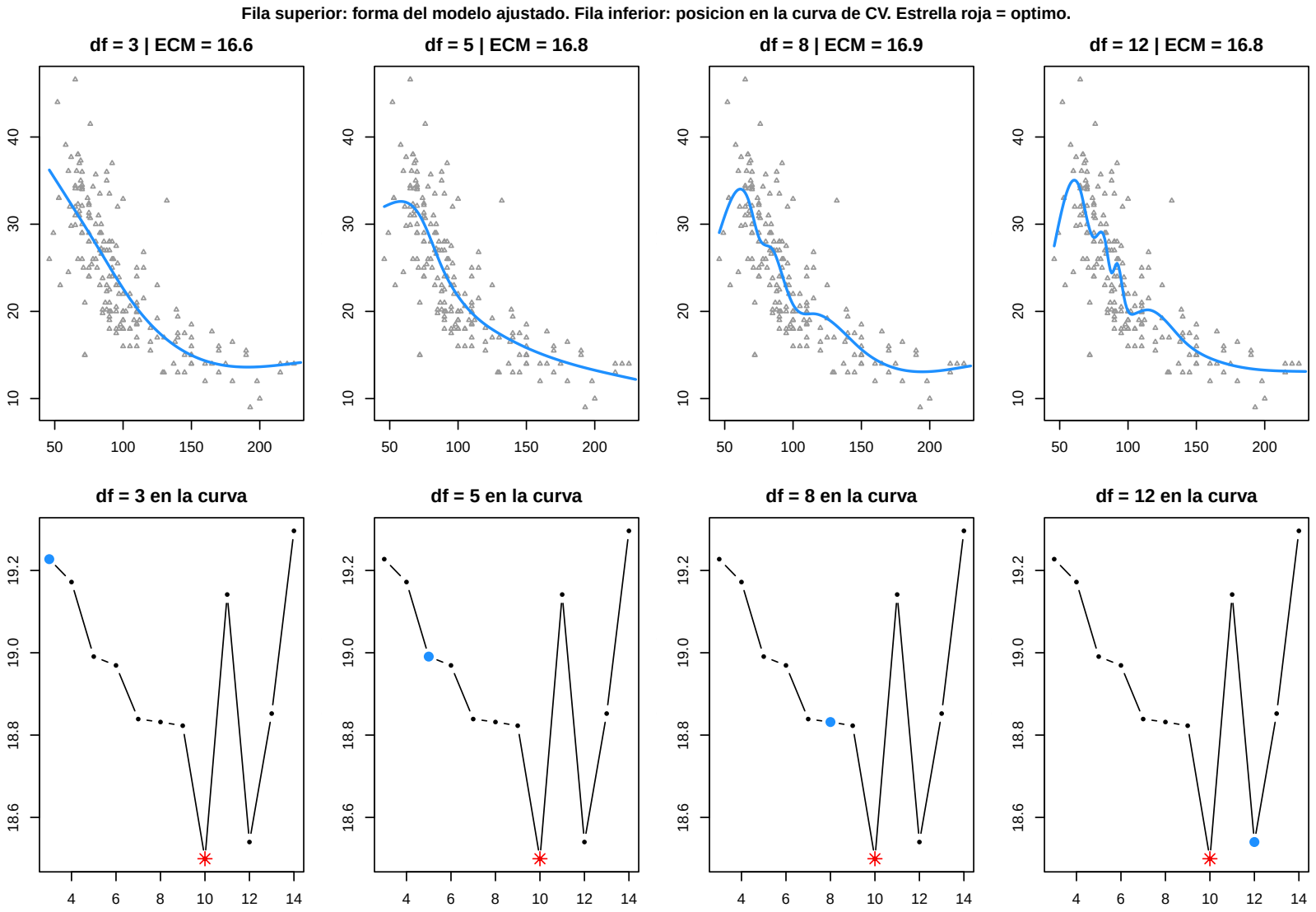
Modelo	df	Que puede ajustar
Lineal (MCO)	2	Altura + inclinacion
Polinomio grado 2	3	+ una curvatura global
Spline cubico, 2 knots	6	+ cambio de curv. en k1 y k2
Spline natural, 2 knots	4	Idem pero lineal en extremos
Smooth spline (λ opt.)	$\approx$ 4–8	Automatico

Mas df: ¿siempre mejor?

Mas df = mas flexibilidad. Pero la flexibilidad tiene un costo:

- **Pocos df:** el modelo es demasiado rigido, no captura la señal real $\Rightarrow$ **subajuste**, ECM_{Test} alto.
- **Demasiados df:** el modelo memoriza el ruido de los datos de entrenamiento $\Rightarrow$ **sobreajuste**, ECM_{Train} bajo pero ECM_{Test} alto.
- **df optimo:** el que minimiza ECM_{Test}. La validacion cruzada es exactamente el mecanismo para encontrarlo, igual que para λ en Ridge/Lasso.
  La seleccion de df por CV conecta directamente con la Sesion 2: el procedimiento es identico.

Forma del modelo ajustado para distintos df

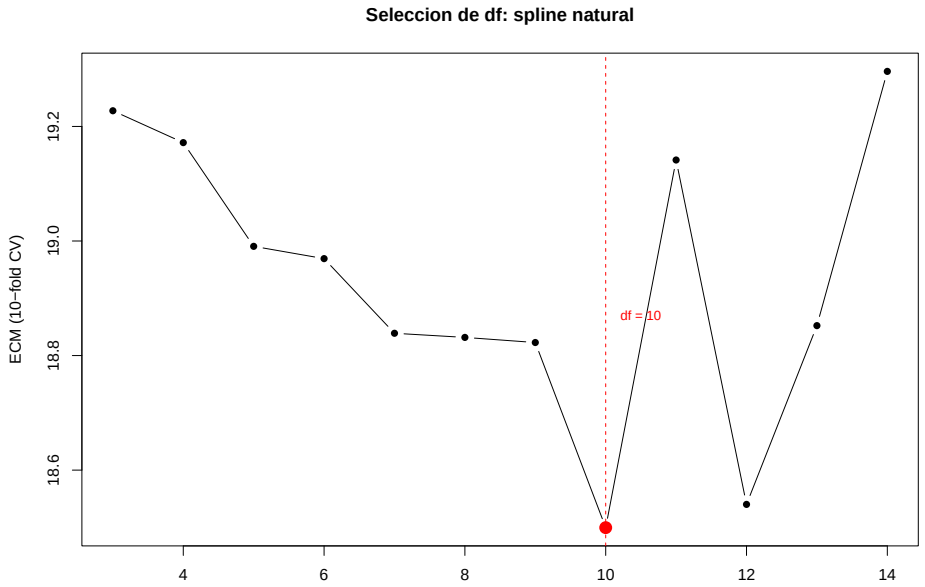


Seleccion de df por validacion cruzada

```
set.seed(1)
mse_cv_ns2 <- NULL ; df_grid2 <- 3:14
for (df_val in df_grid2) {
  glm_fit <- glm(
    mpg ~ ns(horsepower, df=df_val), data=Auto)
  mse_cv_ns2 <- c(mse_cv_ns2,
    cv.glm(Auto, glm_fit, K=10)$delta[1])
}
opt_df2 <- df_grid2[which.min(mse_cv_ns2)]
cat("df optimo:", opt_df2, "\n")
```

```
## df optimo: 10
```

```
plot(df_grid2, mse_cv_ns2, type="b", pch=16,
     xlab="Grados de libertad (df)",
     ylab="ECM (10-fold CV)",
     main="Seleccion de df: spline natural")
points(opt_df2, min(mse_cv_ns2),
       col="red", pch=16, cex=1.8)
abline(v=opt_df2, lty=2, col="red")
text(opt_df2+0.5, min(mse_cv_ns2)*1.02,
     paste("df =", opt_df2), col="red", cex=0.8)
```



Puente: donde estamos

Los problemas de la linealidad

Regresion polinomial

Polinomios truncados y base de funciones

Splines de regresion

Suavizamiento spline

Modelos Aditivos Generalizados (GAMs)

Sintesis y cierre

Referencias

Suavizamiento spline

La limitacion que queda abierta

Los splines de regresion resuelven la discontinuidad, pero la ubicacion y el numero de knots siguen siendo **decisiones subjetivas**. Queremos un metodo que las determine automaticamente.

Idea: usar *todos* los puntos de datos como knots y controlar la flexibilidad penalizando la rugosidad de f:

$$SC_{E_S} = \underbrace{\sum_{i=1}^{n} (y_i - f(x_i))^2}_{\text{ajuste a los datos}} + \lambda \underbrace{\int f''(x)^2 \, dx}_{\text{rugosidad de } f}$$

Interpretacion de los terminos

- **Primer termino**: cuanto se aleja f de los datos.
- **Segundo termino**: cuanto "zigzaguea" la curvatura de f en todo su dominio.
- $\lambda \to 0$: f interpola cada punto (sobreajuste).
- $\lambda \to \infty$: f se convierte en linea recta (MCO).

Analogia con Ridge y Lasso

La penalizacion es analoga a la vista en la Sesion 1:

Metodo	Penalizacion	Controla
Ridge	$\lambda \sum_j \beta_j^2$	Magnitud de β
Lasso	$\lambda \sum_j \lvert\beta_j\rvert$	Dispersion de β
Smooth spline	$\lambda \int f''(x)^2\, dx$	Rugosidad de f

En todos los casos λ controla la complejidad y se selecciona por validacion cruzada.

El vinculo con los splines naturales

Resultado teorico (Representer Theorem): La solucion optima de SC_{E_S} sobre *todas* las funciones suaves posibles es un **spline cubico natural** con knots en cada punto de datos $x_1, \ldots, x_n$.

> *El suavizamiento spline no es una tecnica separada: es un **spline natural con** n **knots** cuya suavidad es controlada por λ en lugar de por el numero de knots. Esto conecta directamente con la seccion anterior.*

Conclusion: no necesitamos buscar la solucion en un espacio enorme de funciones. Sabemos de antemano que la respuesta es un spline natural. Esto hace el problema computable.

Grados de libertad efectivos y la matriz hat

En RLM, la prediccion es lineal en y:

$$\hat{y} = Hy, \quad H = X(X^T X)^{-1} X^T$$

Los df del modelo son $\mathrm{tr}(H) = p$ (numero de columnas de X). Cada 1 en la diagonal de H corresponde a 1 df libre.

En el suavizamiento spline ocurre algo analogo. La prediccion tambien puede escribirse como lineal en y:

$$\hat{y} = \mathbf{S}_\lambda y$$

donde $\mathbf{S}_\lambda$ es la **matriz de suavizamiento**, el analogo de H pero para el modelo penalizado.

Los **df efectivos** son:

$$df_\lambda = \mathrm{tr}(\mathbf{S}_\lambda) \in [2,\, n]$$

Al aumentar λ, $\mathbf{S}_\lambda$ se "aplana'' (se parece mas a la proyeccion sobre una recta) y su traza baja, reflejando menos libertad efectiva.

Ventaja computacional de LOOCV

En RLM, la formula de residuales LOOCV es:

$$e_i^{\text{LOO}} = \frac{y_i - \hat{y}_i}{1 - h_{ii}}$$

donde h_{ii} es el i-esimo elemento diagonal de H. Esto calcula el ECM_{LOOCV} de los n datos con **un solo ajuste**.

Para el smooth spline, la formula es identica reemplazando h_{ii} por S_{ii} (diagonal de $\mathbf{S}_\lambda$):

$$ECM_{\text{LOOCV}} = \frac{1}{n} \sum_{i=1}^{n} \left(\frac{y_i - \hat{f}_\lambda(x_i)}{1 - S_{ii}} \right)^2$$

Ventaja clave

Se mide la capacidad de prediccion dando de baja a cada uno de los n puntos, pero ajustando el modelo **una sola vez**. No hay que refitear n veces como en el LOOCV general.

GCV y REML: alternativas para seleccionar lambda

Calcular S_{ii} puede ser costoso. Existen dos alternativas:

GCV – Validacion Cruzada Generalizada: Reemplaza cada S_{ii} por el promedio $\mathrm{tr}(\mathbf{S})/n$, asumiendo que todos los puntos tienen igual influencia:

$$GCV(\lambda) = \frac{n \cdot RSS(\lambda)}{(n - \mathrm{tr}(\mathbf{S}_\lambda))^2}$$

Computacionalmente simple. Tiende a elegir modelos ligeramente mas flexibles. Es el metodo en `smooth.spline(..., cv=FALSE)`.

REML – Maxima Verosimilitud Restringida: Trata cada funcion suave f_j como un efecto aleatorio y λ_j como su varianza. Maximiza una funcion de verosimilitud que descuenta los efectos fijos.

Conceptualmente: hace un balance probabilistico riguroso entre ajuste y rugosidad. Es el metodo por defecto en `mgcv::gam()` y generalmente produce mejores estimaciones, especialmente con varios suavizadores simultaneos.

Implementacion: smooth.spline()

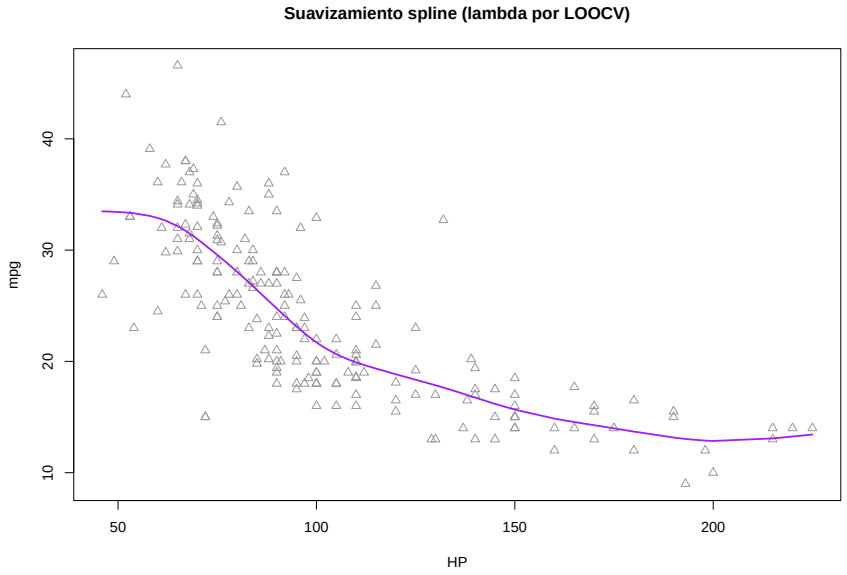
```
mod_ss <- smooth.spline(
  Auto$horsepower[train],
  Auto$mpg[train],
  cv = TRUE)  # cv=TRUE: LOOCV; cv=FALSE: GCV

cat("Lambda seleccionado:", round(mod_ss$lambda, 5), "\n")
```

```
## Lambda seleccionado: 0.00158
```

```
cat("df efectivos:       ", round(mod_ss$df,     2),  "\n")
```

```
## df efectivos:        7.08
```

```
plot(Auto$horsepower[train], Auto$mpg[train],
     pch=2, col="grey60", xlab="HP", ylab="mpg",
     main="Suavizamiento spline (lambda por LOOCV)")
lines(mod_ss, col="purple", lwd=2)
```



ECM de prueba del suavizamiento spline

```
mse_ss <- mean(
  (Auto$mpg[test] -
     predict(mod_ss, Auto$horsepower[test])$y)^2)
cat("ECM prueba:", round(mse_ss, 2),
    "| df efectivos:", round(mod_ss$df, 2))
```

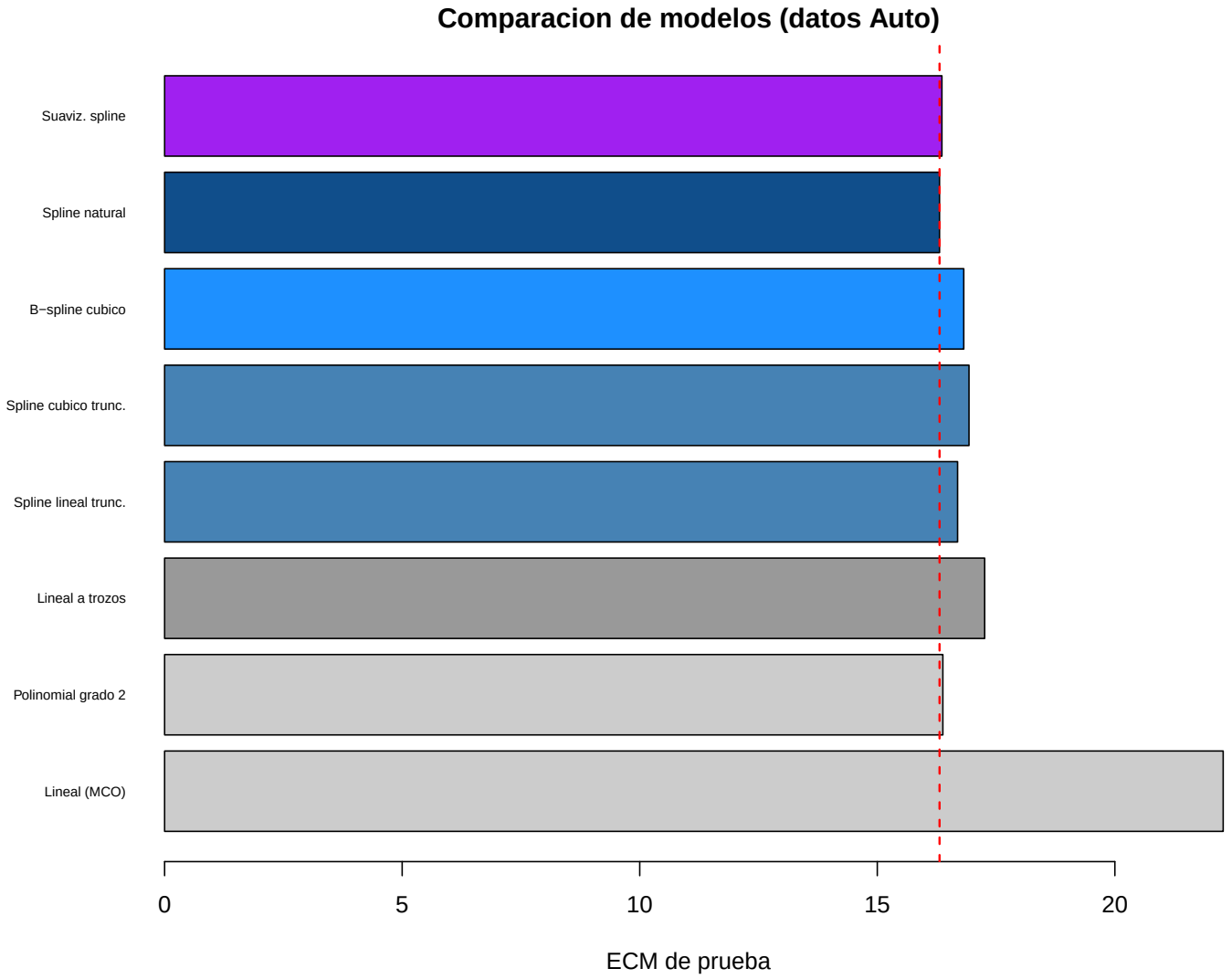
```
## ECM prueba: 16.36 | df efectivos: 7.08
```

Menor ECM sin especificar knots manualmente. La flexibilidad fue seleccionada automaticamente por LOOCV.

Conexion con el Taller 1

En el taller, Lasso selecciona genes bajo linealidad. Si un gen clave tiene efecto curvo, el suavizamiento spline permite modelarlo: el mismo mecanismo de CV del taller (ya aplicado para encontrar λ_{Lasso}) se usa aqui para encontrar λ del spline.

Comparacion completa de modelos



La linea roja marca el menor ECM. Los modelos con penalizacion automatica (spline de suavizamiento) alcanzan menores ECM sin decisiones subjetivas sobre knots.

Tabla resumen

Table 4: Comparacion de modelos – datos Auto

Modelo	ECM Prueba	df
Lineal (MCO)	22.28	2.0
Polinomial grado 2	16.38	3.0
Lineal a trozos	17.26	4.0
Spline lineal trunc.	16.69	4.0
Spline cubico trunc.	16.93	6.0
B-spline cubico	16.82	6.0
Spline natural	16.31	4.0
Suaviz. spline	16.36	7.1

Cada modelo reduce el ECM al ganar flexibilidad relevante. Mas df no siempre gana: la clave es que la flexibilidad adicional capture señal real, no ruido.

Puente: donde estamos

Los problemas de la linealidad

Regresion polinomial

Polinomios truncados y base de funciones

Splines de regresion

Suavizamiento spline

Modelos Aditivos Generalizados (GAMs)

Sintesis y cierre

Referencias

Modelos Aditivos Generalizados (GAMs)

Las variables del ejemplo

Usamos el dataset `Auto`: 392 autos de los anos 70–80.

- **mpg** (*miles per gallon*): cuantas millas recorre el carro por galon de gasolina. **Variable respuesta.** Mayor mpg = mas eficiente en combustible.
- **horsepower**: potencia del motor. Cuanta fuerza tiene para mover el carro. Mayor HP = mas rapido pero generalmente menos eficiente.
- **weight**: peso del carro en libras. Mas pesado = necesita mas energia = generalmente menor mpg.
- **displacement**: cilindrada en pulgadas cubicas. El volumen total de los cilindros del motor. Motor mas grande = mas potencia pero generalmente mas consumo.
- **cylinders**: numero de cilindros (las "camaras'' donde se quema la gasolina). Tipicamente 4, 6 u 8 cilindros. Mas cilindros = mas potencia = usualmente menos eficiencia.

La extension a multiples covariables

Hasta ahora: una sola covariable (x = horsepower). En la practica hay p variables. El modelo lineal las combina como $\beta_1 x_1 + \cdots + \beta_p x_p$, asumiendo linealidad en cada una.

Si queremos no-linealidad en cada variable y mantenemos la **aditividad**, obtenemos un **GAM**:

$$y_i = \beta_0 + f_1(x_{i1}) + f_2(x_{i2}) + \cdots + f_p(x_{ip}) + \epsilon_i$$

Relacion con los modelos anteriores

- Si $f_j(x_j) = \beta_j x_j$: recuperamos RLM.
- Si cada f_j es un smooth spline: GAM no parametrico.
- Los GAMs generalizan tanto RLM como los splines univariados.

Estimacion: algoritmo de backfitting

La funcion penalizada del GAM es:

$$SC_{GAM} = \sum_i \left(y_i - \beta_0 - \sum_j f_j(x_{ij}) \right)^2 + \sum_j \lambda_j \int f_j''(x)^2 \, dx$$

Se minimiza con el algoritmo de **backfitting**:

1. Inicializar $\hat{f}_j \equiv 0$ para todo j.
2. Calcular **residuales parciales** para j: $r_{ij} = y_i - \hat{\beta}_0 - \sum_{k \neq j} \hat{f}_k(x_{ik})$
3. Ajustar $\hat{f}_j$ como smooth spline de r_{ij} sobre x_j.
4. Repetir pasos 2–3 para cada j, hasta convergencia.

Cada λ_j se selecciona automaticamente por REML o GCV. El resultado es una curva $\hat{f}_j$ por variable, con su propio nivel de suavidad.

GAM en R: ajuste y resumen

```r
library(mgcv)
mod_gam <- gam(
  mpg ~ s(horsepower, bs="cs") +
        s(weight,     bs="cs"),
  data=Auto[train,])
mse_gam2 <- mean(
  (Auto$mpg[test]-predict(mod_gam,Auto[test,]))^2)
cat("ECM (GAM 2 vars):", round(mse_gam2,2), "\n\n")
```

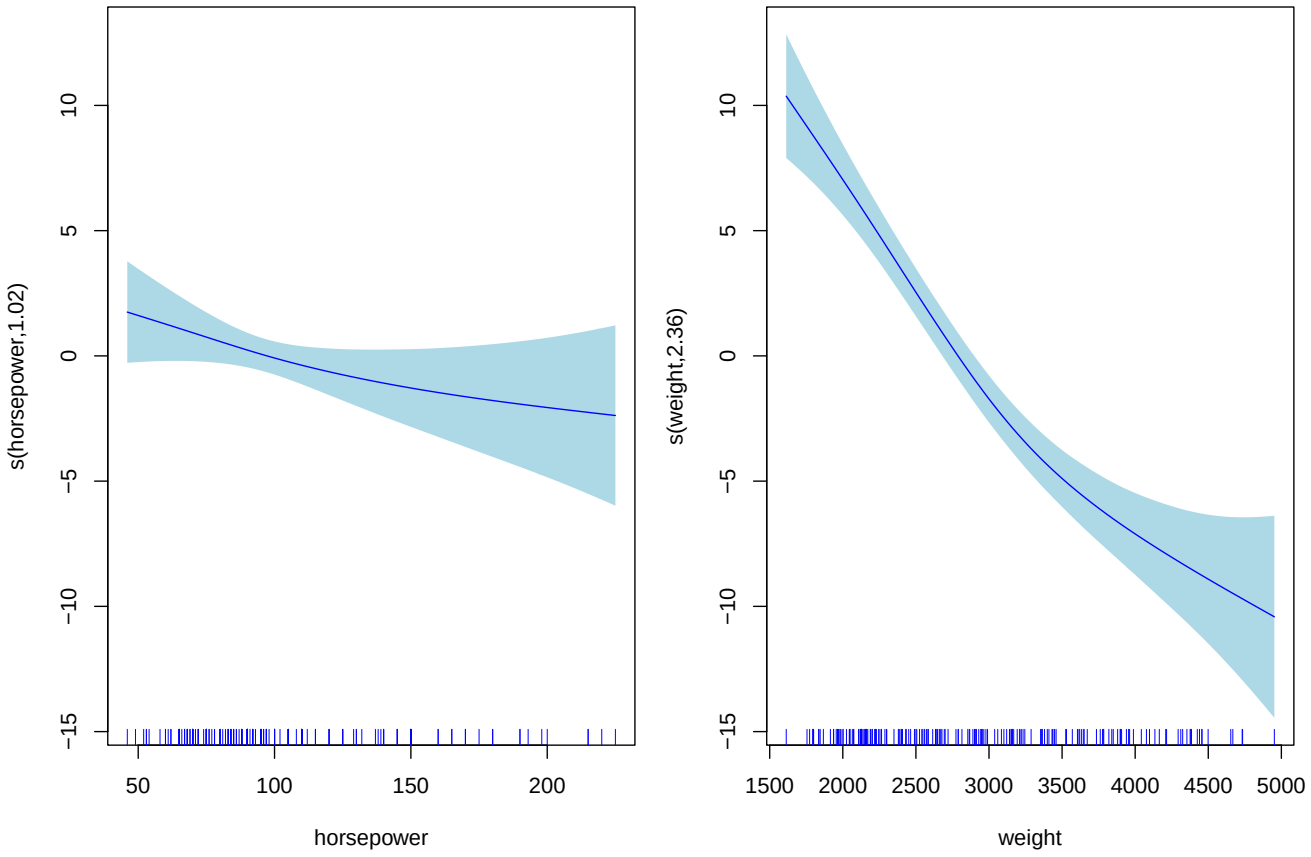
```
## ECM (GAM 2 vars): 16.95
```

```r
# edf: df estimados por variable.
# edf=1 equivale a efecto lineal (recta)
summary(mod_gam)$s.table
```

```
##                     edf Ref.df          F    p-value
## s(horsepower) 1.020485      9  0.3973671 0.02623969
## s(weight)     2.361028      9 14.1676844 0.00000000
```

Cuando `edf` es cercano a 1, el efecto de esa variable es aproximadamente lineal y un termino simple $\beta_j x_j$ podria ser suficiente.

Efectos parciales: interpretacion



Cada panel muestra $\hat{f}_j(x_j)$ manteniendo las demas variables constantes (**efecto parcial**). La banda sombreada es el IC puntual de $\hat{f}_j$.

Si la banda incluye la linea horizontal 0, la variable podria no aportar (analogo al valor-p en RLM).

GAM con multiples covariables

```
mod_gam_full <- gam(
  mpg ~ s(horsepower,   bs="cs") +
        s(weight,       bs="cs") +
        s(displacement, bs="cs") +
        cylinders,
  data=Auto[train,])
mse_gam_full <- mean(
  (Auto$mpg[test]-predict(mod_gam_full,Auto[test,]))^2)
cat("ECM (GAM completo):   ", round(mse_gam_full,2), "\n")
```

```
## ECM (GAM completo):    16.62
```

```
cat("ECM (smooth spline):  ", round(mse_ss,       2), "\n")
```

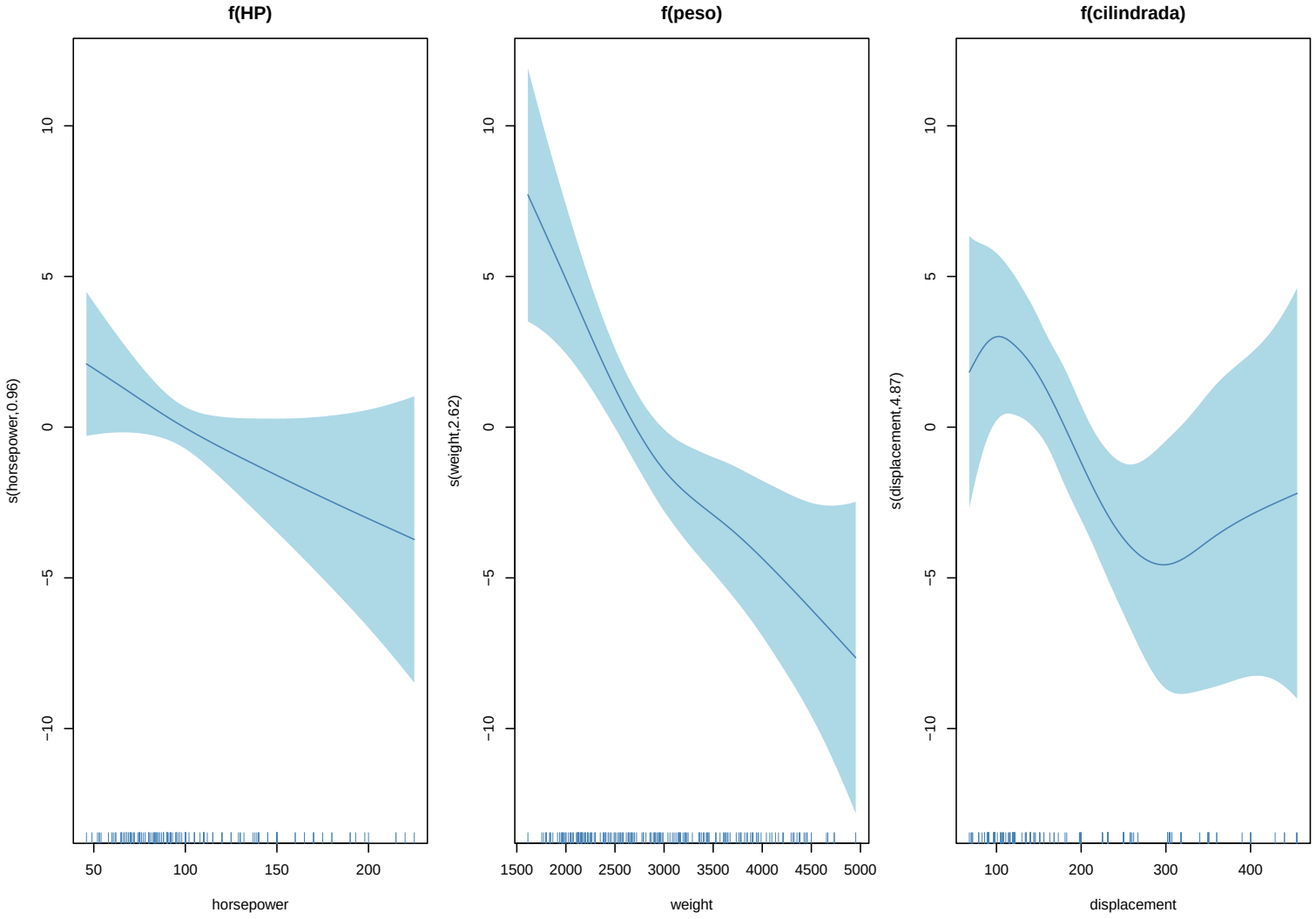
```
## ECM (smooth spline):   16.36
```

```
cat("ECM (polinomial d=2): ", round(mse2,         2), "\n")
```

```
## ECM (polinomial d=2):  16.38
```

`cylinders` entra de forma lineal: tiene solo valores discretos (4, 6, 8), por lo que `s()` no es apropiado.

Efectos parciales: GAM completo



Cada curva captura una relacion no lineal distinta. El peso, por ejemplo, muestra un efecto aproximadamente lineal (curva casi recta), mientras que HP y cilindrada tienen curvatura mas pronunciada.

Ventajas de los GAMs

- **Interpretabilidad:** se puede visualizar el efecto marginal de cada variable por separado con su IC.
- **Flexibilidad automatica:** la no-linealidad de cada f_j se ajusta por REML/GCV sin decidir knots.
- **Extensibles:** cambiando la funcion de enlace se puede usar con respuestas binarias (logistico), de conteo (Poisson), de supervivencia, etc.
- **Seleccion de variables:** en `mgcv` se puede penalizar $f_j \equiv 0$, analogo al Lasso.

Limitaciones de los GAMs

- **No modelan interacciones** entre variables por construccion. Se pueden añadir manualmente con `te()` o `ti()` en `mgcv`, pero el modelo pierde aditividad y se vuelve mas complejo.
- **Con** $p \gg n$ (como en el Taller) requieren seleccion previa de variables. Un GAM directo con 5000 genes es inviable.
- **La aditividad puede ser restrictiva** si las variables interactuan fuertemente. En ese caso, arboles de decision o random forest (Sesion 4) son preferibles.

Alta dimension + no linealidad: el pipeline

Pregunta practica del Taller: con 5000 genes, no podemos ajustar directamente un GAM. ¿Como procedemos si sospechamos efectos no lineales?

Pipeline estandar en dos etapas:

Etapa 1 – Seleccion (Lasso): Lasso identifica los genes con alguna relacion con la respuesta. Incluso si el efecto verdadero es curvo, Lasso asignara un coeficiente no nulo (la recta es la mejor aproximacion lineal a cualquier curva). Los genes con efecto nulo reciben $\hat{\beta} = 0$.

Etapa 2 – Modelado no lineal (GAM/splines): con los $p^* \ll p$ genes seleccionados, se ajusta un GAM sobre ese subconjunto de baja dimension. Los metodos de esta sesion son directamente aplicables.

Alta dimension + no linealidad: justificacion

Por que Lasso es robusto a la no-linealidad

Un gen con efecto curvo (no lineal) producira un coeficiente Lasso no nulo: la recta es la mejor aproximacion lineal a la curva y captura parte de la señal. El riesgo es que Lasso subestime la importancia del gen, pero rara vez lo excluye completamente si el efecto es real.

Alternativa avanzada: el paquete `gamsel` implementa un Lasso dentro de un GAM, realizando seleccion de variables y no-linealidad en un solo paso. Computacionalmente mas costoso y mas dificil de interpretar.

Para el Taller 1: Lasso selecciona los genes relevantes (preguntas 3–6). Si en el analisis del punto 8 se observan patrones curvos en los residuales del modelo Lasso, un GAM sobre los genes seleccionados es el siguiente paso natural.

Puente: donde estamos

Los problemas de la linealidad

Regresion polinomial

Polinomios truncados y base de funciones

Splines de regresion

Suavizamiento spline

Modelos Aditivos Generalizados (GAMs)

Sintesis y cierre

Referencias

Sintesis y cierre

Mapa conceptual de la sesion

Metodo	Continua	Suave	Knots	Multi-X
Lineal (MCO)	No	–	–	Si
Polinomio global	Si	Si	–	No
Piezas (a trozos)	No	No	Manual	No
Spline lineal	Si	No	Manual	No
Spline cubico	Si	Si	Manual	No
Spline natural	Si	Si	Manual	No
Suaviz. spline	Si	Si	Automatico	No
GAM	Si	Si	Automatico	Si

Hilo conductor del curso

Sesion 1: MCO – problemas con $p \gg n$ – Ridge y Lasso.

Sesion 2: Como seleccionar λ – validacion externa.

Sesion 3: Y si la relacion no es lineal – Splines y GAMs.

Los tres temas comparten la misma estructura:

1. Identificar la limitacion del modelo estandar.
2. Proponer una extension penalizada o flexible.
3. Seleccionar la complejidad mediante validacion externa.

Guia practica: que metodo usar

Situacion	Metodo
Relacion lineal confirmada	MCO / Ridge / Lasso
No-linealidad leve, 1 variable	Polinomio grado 2–3
No-linealidad, knots conocidos	Spline cubico `bs()`
No-linealidad, extremos inestables	Spline natural `ns()`
No-linealidad, sin knots previos	Suavizamiento spline
Varias variables, efectos aditivos	GAM `mgcv::gam()`
Alta dimension + no-linealidad	Lasso entonces GAM
Interacciones entre variables	Sesion 4 (arboles)

Puente: donde estamos

Los problemas de la linealidad

Regresion polinomial

Polinomios truncados y base de funciones

Splines de regresion

Suavizamiento spline

Modelos Aditivos Generalizados (GAMs)

Sintesis y cierre

Referencias

Referencias

Bibliografia

(1) Hastie, T., Tibshirani, R., Friedman, J. *The Elements of Statistical Learning*. Springer. (Cap. 5 y 9)

(2) James, G., Witten, D., Hastie, T., Tibshirani, R. *An Introduction to Statistical Learning with R*, 2a ed. Springer. (Cap. 7)

(3) Wood, S. N. *Generalized Additive Models: An Introduction with R*, 2a ed. CRC Press.

Paquetes en R:

- `splines::bs()`, `splines::ns()` – B-splines y splines naturales.
- `stats::smooth.spline()` – Suavizamiento spline (LOOCV/GCV).
- `mgcv::gam()` – GAMs con REML/GCV automatico.
- `glmnet::glmnet()` – Ridge/Lasso (Sesiones 1–2).
- `gamsel` – Lasso + GAM para alta dimension.